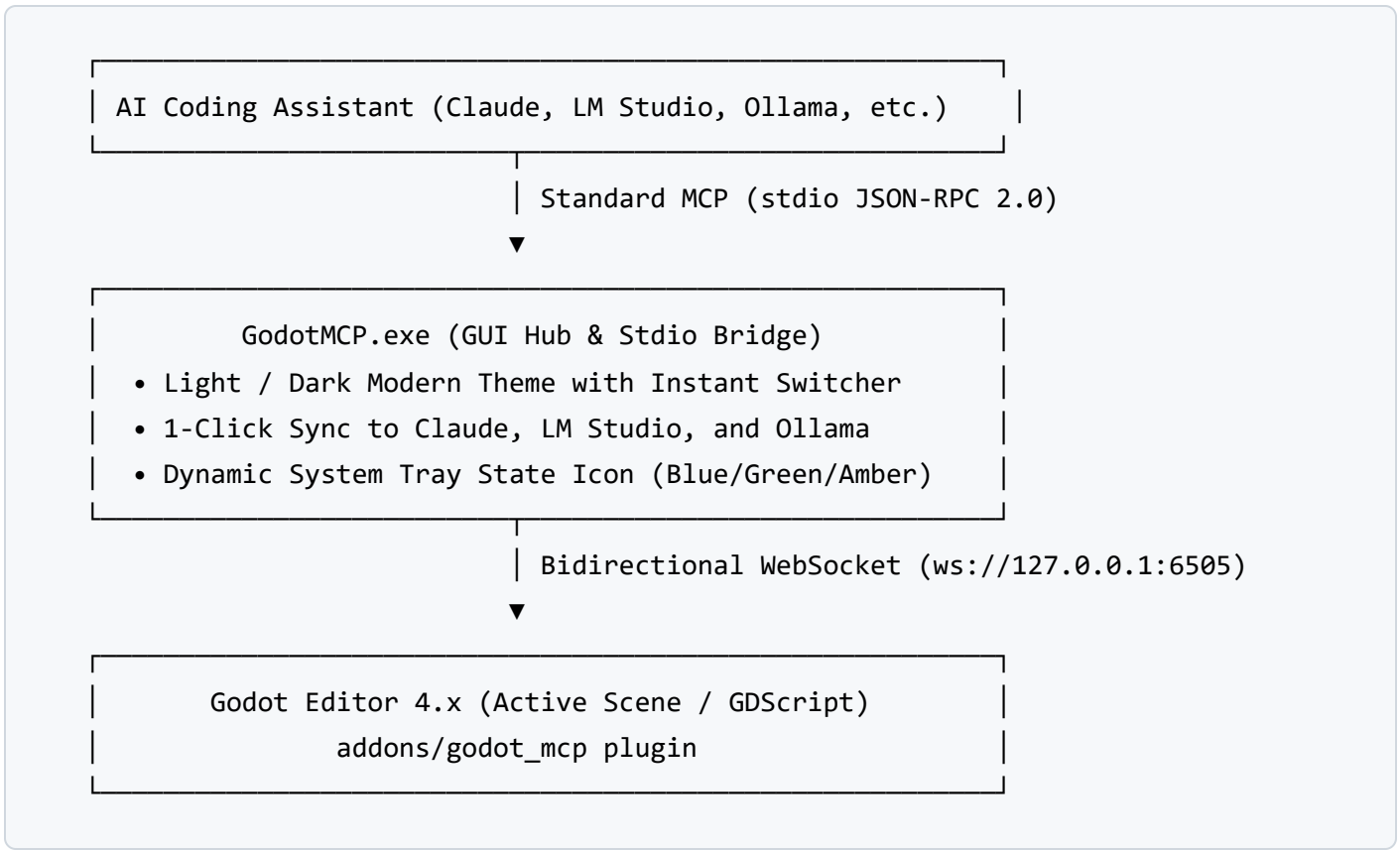
Godot-MCP — Model Context Protocol Server & GUI Hub for Godot Engine 4.x

(Community Edition — Free & Open Source under MIT)



Godot-MCP (Community Edition) is an open-source, portable desktop GUI hub and Model Context Protocol (MCP) server that connects local and cloud AI assistants (**Claude Desktop / Code**, **LM Studio**, **Ollama**, **Cursor**, **Antigravity**) directly into the active **Godot Engine 4.x Editor**.



Highlights

- **Dual-Mode Executable**:
- **Double-Click**: Launches the modern WPF desktop hub with Light/Dark mode switcher, project management, and 1-click AI assistant synchronization.
- **Invoked by AI Client**: Runs silently and headlessly over stdio as a JSON-RPC 2.0 server without popping up windows.
- **Light & Dark Theme Switcher**: Defaults to a clean, crisp Light Mode (`#F5F5F7`), with a 1-click top-header toggle to Dark Mode (`#1E1E1E`), persisted in user preferences.
- **Segmented Pill Navigation Bar**: Seamless tab-less switching across 💬 **Local AI Chat**, 🎮 **Godot Workspace**, 🚀 **Publish & Ship**, and ✨ **Pro Ecosystem**.
- **Godot Asset Library Distribution**: Built-in 1-click syntax validator, local folder browser, and real .zip archive packager generating AssetLib-compliant distribution bundles.
- **Dedicated VRAM Eviction Controls (Nuclear Option)**: Permanent toolbar buttons ⏏ Eject Model and 🧹 Purge All VRAM to instantly purge loaded models from GPU VRAM across both Ollama and LM Studio (lms unload --all).
- **Single-Instance Lifecycle Enforcement**: Kernel mutex and event wait handle prevent duplicate background instances and eliminate lingering ghost tray icons on application exit.
- **Dynamic Hexagonal Tray Icon**:
- 🔵 **Blue**: Waiting for Godot Editor to connect on port 6505.
- 🟢 **Green**: Godot 4 Editor is connected live.
- 🟡 **Amber**: AI model is actively executing a command inside Godot.
- **1-Click AI Client Sync**: Automatically configures `claude_desktop_config.json`, LM Studio's `mcp.json`, and Ollama's `mcp.json` with automatic environment variable injection (`"env": { ... }`).
- **Hardware Tuning & Custom Environment Options**: Fine-tune timeouts and parameters (`MCP_OLLAMA_TIMEOUT_SECONDS`, `MCP_LMSTUDIO_TIMEOUT_SECONDS`, `MCP_AI_CHAT_TIMEOUT_SECONDS`, `GODOT_PORT`) with Registry persistence and 1-click presets.
- **Native Window Focus Handshake**: Single-instance mutex with Win32 `AllowSetForegroundWindow` permission handoff smoothly brings existing running Hub instances to the front.
- **Live Godot 4 Integration**: Live scene tree inspector, real-time node addition/deletion, property updates, viewport screenshot capture, and GDScript execution.
- **Pro Suite Showcase**: Discover the broader MCP Suite Pro ecosystem (Blender, Aseprite, Krita, Audacity, FMOD, Wwise, Spine, DaVinci Resolve) directly from the lower deck.

Quickstart

Option 1: Download from itch.io (Pre-Compiled Binary)

Download the standalone `GodotMCP.exe` binary from **itch.io**. No .NET runtime installation is required—it is 100% self-contained.

Option 2: Build from Source

```
git clone https://github.com/Unsigned-Exe/Godot-MCP-Community-Edition.git
cd Godot-MCP-Community-Edition/src
dotnet publish GodotMCP.csproj -c Release -r win-x64 --self-contained -p:PublishSingleF
```

Option 3: Open & Build with Visual Studio

This repository includes a pre-configured solution file (`GodotMCP.sln`) for opening and compiling the project directly inside Visual Studio 2022 or 2026:

1. **Clone the Repository**: `bash git clone https://github.com/Unsigned-Exe/Godot-MCP-Community-Edition.git`
2. **Open the Solution**: Double-click `GodotMCP.sln` to open it in Visual Studio (requires the **.NET Desktop Development** workload).
3. **Build the Solution**:
4. Set configuration to **Release** and platform to **Any CPU** or **x64**.
5. Press `Ctrl+Shift+B` (or select **Build -> Build Solution**).
6. The compiled binary will be placed at `dist/GodotMCP.exe` (or `src/bin/Release/net8.0-windows/GodotMCP.exe`).

Usage

1. Launch the Desktop Hub

Double-click `GodotMCP.exe` to launch the GUI dashboard. - Select your Godot project directory. - Click "⚡ **Install Addon in Project**". - Click "**▶ Launch Godot Editor**". - In Godot: Navigate to **Project -> Project Settings -> Plugins** and check **Enable** next to `Godot-MCP`. - Click "🔄 **Sync Selected Assistants**" to configure Claude, LM Studio, or Ollama in one click.

2. Submitting Your Addon to the Godot Asset Library

Godot-MCP includes a built-in pre-flight packager in the 🚀 **Publish & Ship** tab to prepare your addon for the official Godot Asset Library:

1. **Select Project**: In the 🎮 **Godot Workspace** tab, ensure your Godot project folder is selected.
2. **Open Publish & Ship**: Switch to the 🚀 **Publish & Ship** tab.
3. **Verify Git URL / Directory**: Verify your public Git repository URL (or click **Browse...** to pick a local distribution directory).
4. ⚡ **Validate & Package**: Click **Validate & Package**. Godot-MCP will:
5. Verify addons/godot_mcp/plugin.cfg syntax and version declarations.
6. Confirm Godot 4.x compatibility.
7. Automatically compile a clean, AssetLib-compliant distribution archive at /dist/godot_mcp_assetlib.zip.
8. **Submit to Godot AssetLib**:
9. Click 🌐 **Open AssetLib Portal** (or visit godotengine.org/asset-library/asset).
10. Log into your Godot account and click **Submit Asset**.
11. Fill in:
    - **Asset Name**: Godot-MCP (or your addon name)
    - **Category**: Tools / Plugins
    - **Godot Version**: 4.x
    - **Repository URL**: Your public Git URL (e.g. https://github.com/Unsigned-Exe/Godot-MCP-Community-Edition)
    - **Download Tag/Commit**: 1.0.0 or main
12. Click **Submit for Review**. Once approved by Godot moderators, any developer can install your addon directly from inside the Godot Editor.

3. Command-Line Options

```
GodotMCP.exe [options]

OPTIONS:
  -h, --help                 Display help information.
  -v, --version              Display application version.
  -p, --port <port>          Specify WebSocket port for Godot Editor (default: 6505).
  --server                   Run headlessly as stdio MCP server for AI clients.
  --tray                     Start minimized to the Windows system tray.
  --install-addon <path>     Installs the godot_mcp plugin into a target project.
```

Manual AI Client Configuration

If you prefer manual configuration, add this block to your client:

Claude Desktop (`%APPDATA%\Claude\claude_desktop_config.json`)

```
{
  "mcpServers": {
    "godot-4-7": {
      "command": "C:\\path\\to\\GodotMCP.exe",
      "args": ["--server"]
    }
  }
}
```

LM Studio (`%USERPROFILE%\.cache\lm-studio\mcp.json`)

```
{
  "mcpServers": {
    "godot-4-7": {
      "command": "C:\\path\\to\\GodotMCP.exe",
      "args": ["--server"]
    }
  }
}
```

Ollama (`%USERPROFILE%\.ollama\mcp.json`)

```
{
  "mcpServers": {
    "godot-4-7": {
      "command": "C:\\path\\to\\GodotMCP.exe",
      "args": ["--server"]
    }
  }
}
```

Tool Reference (25 Tools)

Tool Name	Description
`godot_get_status`	Returns Godot Editor WebSocket connection state, port, and engine binary.
`godot_create_project`	Programmatically creates a new Godot 4 project directory with `project.godot`.
`godot_launch_editor`	Launches the Godot 4 Editor for the specified project.
`godot_run_project`	Runs and plays the active game project.
`godot_check_syntax`	Performs fast headless GDScript syntax validation (`--check-only`).
`godot_read_file`	Reads file text content from the project directory.
`godot_write_file`	Writes or updates arbitrary project files.
`godot_delete_file`	Safely removes files along with their `.import` and `.uid` metadata.
`godot_list_project_files`	Enumerates project assets and scripts, ignoring cache folders.
`godot_write_script`	Creates or replaces a `.gd` script.
`godot_edit_script_snippet`	Replaces an isolated snippet in an existing GDScript file.
`godot_append_to_script`	Appends code cleanly to the end of a GDScript file.
`godot_create_scene`	Generates a valid `.tscn` text scene file with designated root node type.
`godot_execute_editor_script`	Executes arbitrary GDScript live inside the running Godot Editor.
`godot_get_scene_tree`	Fetches the complete node hierarchy of the active editor scene tab.
`godot_add_node`	Adds a child node live to the currently open scene tree.

Tool Name	Description
`godot_delete_node`	Removes a node live from the current scene.
`godot_update_node_property`	Edits any property on an active node in real time.
`godot_save_active_scene`	Saves the open scene tab in the editor.
`godot_open_scene`	Opens a designated `.tscn` file into an editor tab.
`godot_play_current_scene`	Plays the currently active scene in a game window.
`godot_stop_running_scene`	Stops the running scene process.
`godot_get_editor_errors`	Reads the latest debugger and error output from the editor panel.
`godot_get_editor_screenshot`	Captures a high-resolution screenshot of the Godot editor viewport.
`godot_call_editor_command`	Dispatches arbitrary extension methods to the `godot_mcp` plugin.

Disclaimers & Hardware Safety

Godot MCP coordinates high-frequency tool invocations and local AI models (such as Ollama or LM Studio) which can place sustained, intensive computational loads on host hardware. The software is provided **"AS IS"** without warranties of any kind.

Please review the full Hardware Damage & Workload Disclaimer regarding GPU thermal safety, power delivery requirements, and limitations of liability.

License

This project is licensed under the **MIT License** — see the LICENSE file for details.

Developed with ❤️ by **Unsigned Exe Studios** (unsignedexe.com).

Disclaimers & Limitations of Liability

> Godot-MCP (Community Edition) | Unsigned Exe Studios

1. "As-Is" Disclaimer of Warranties

THE SOFTWARE IS PROVIDED "AS IS" AND "AS AVAILABLE", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, AND NON-INFRINGEMENT. AS REFLECTED IN THE MIT LICENSE, THE ENTIRE RISK AS TO THE QUALITY, PERFORMANCE, AND OPERATION OF THE SOFTWARE RESTS ENTIRELY WITH THE USER.

2. High-Stress Workloads & Hardware Damage Disclaimer

CERTAIN INTEGRATED WORKFLOWS, INCLUDING BUT NOT LIMITED TO LOCAL AI/LLM INFERENCE (VIA OLLAMA, LM STUDIO, OR EMBEDDED ENGINE HOOKS), 3D SCENE GENERATION, MASS ASSET SCRIPTING, AND CONTINUOUS TOOL EXECUTION LOOPS, PLACE INTENSIVE, SUSTAINED DEMANDS ON SYSTEM HARDWARE.

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL UNSIGNED EXE STUDIOS, ITS AUTHORS, OR CONTRIBUTORS BE LIABLE FOR ANY HARDWARE DAMAGE OR COMPONENT FAILURE WHATSOEVER, INCLUDING BUT NOT LIMITED TO: * **THERMAL DEGRADATION, SILICON BURNOUT, OR OVERHEATING** OF GRAPHICS PROCESSING UNITS (GPUs), CENTRAL PROCESSING UNITS (CPUs), OR HARDWARE ACCELERATORS; * **POWER SUPPLY UNIT (PSU) OR VOLTAGE REGULATOR MODULE (VRM) FAILURES** RESULTING FROM SUSTAINED POWER DRAW OR TRANSIENT LOAD SPIKES; * **MOTHERBOARD, CHIPSET, VRAM, OR SYSTEM MEMORY INSTABILITY OR DEFECTS**; * **STORAGE DRIVE (SSD/NVMe/HDD) WEAR, DATA LOSS,**

CORRUPTION, OR FILE SYSTEM DAMAGE; * **ACOUSTIC, FAN, OR COOLING APPARATUS WEAR OR DAMAGE**.

THE USER EXPRESSLY ACKNOWLEDGES AND ASSUMES ALL RISKS ASSOCIATED WITH RUNNING HIGH-COMPUTE WORKLOADS. IT IS THE USER'S SOLE RESPONSIBILITY TO MAINTAIN ADEQUATE SYSTEM COOLING, VERIFY THERMAL OPERATING LIMITS, CONFIGURE PROPER FAN CURVES, PROVIDE SUFFICIENT POWER DELIVERY, AND MAINTAIN COMPREHENSIVE DATA BACKUPS.

---

3. Limitation of Liability

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, UNDER NO CIRCUMSTANCES SHALL UNSIGNED EXE STUDIOS OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, PUNITIVE, OR CONSEQUENTIAL DAMAGES (INCLUDING LOSS OF PROFITS, LOSS OF BUSINESS, LOSS OF DATA, OR INTERRUPTED PRODUCTION), ARISING FROM ANY CAUSE OR LEGAL THEORY (WHETHER IN CONTRACT, STRICT LIABILITY, TORT INCLUDING NEGLIGENCE, OR OTHERWISE), EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.